

هدف ریفکتورینگ "کوتاه‌ترین کد ممکن" نیست، بلکه "خوانا‌ترین کد ممکن" هست.

کد باید طوری باشه که یه برنامه‌نویس دیگه (یا خودت ۶ ماه بعد) توی ۳۰ ثانیه بفهمه چیکار می‌کنه.

نسخه متعادل این مزیت‌ها رو داره:

- ✓ اسم متغیرها مثل مدرک عمل می‌کنن و جریان کار رو نشون میدن
- ✓ موقع Debug می‌تونن ببینن هر بخش جداگانه چه مقداری داره
- ✓ هنوز Long Method نیست (فقط ۶ خطه)

پس حق با توئه - نسخه خیلی فشرده یه کم گنگ شده. نسخه متعادل بهتره. 🧠

بریم سراغ **Introduce Parameter Object**؟

بریم

چشم، بفرما لینکش رو:

<https://refactoring.guru/introduce-parameter-object>

منتظرم تا صفحه رو ببینیم و با هم کارش کنیم. 🚀

<https://refactoring.guru/introduce-parameter-object>

.Link reading is unavailable in Expert Mode. Please use Instant Mode

Introduce Parameter Object

تعریف ساده

وقتی یه متد چندین پارامتر داره که همیشه با هم میان و میرن (یا به هم مرتبطن)، یه کلاس جدید بساز و این پارامترها رو توی اون کلاس جمع کن. بعدش به جای چندتا پارامتر، فقط یه شیء از اون کلاس رو به متد پاس بده.

چرا این کار رو می‌کنیم؟ 🧠

مشکل پارامترهای پراکنده	توضیح
Long Parameter List	خودش به بوی بد دیگه ست - متدی با ۳-۴ تا پارامتر یا بیشتر
Data Clumps	یه گروه داده که همیشه با هم سفر می‌کنن ولی خونه (کلاس) ندارن
تغییرات سخت	اگه یه فیلد جدید اضافه بشه، باید امضای همه متدها رو عوض کنی
کد تکراری	احتمالاً اعتبارسنجی این داده‌ها چندجا تکرار شده

مثال قبل از ریفکتور (مسئله واقعی)

فرض کن به سرویس پیامک داری:

csharp

```

public class NotificationService
{
    ,public void SendSms(string phoneNumber, string message
    ,string senderName, DateTime sendDate
    (bool isUrgent, string language

// اعتبارسنجی پارامترها
if (string.IsNullOrEmpty(phoneNumber))
    throw new ArgumentException("شماره موبایل الزامیست");
    if (string.IsNullOrEmpty(message))
        throw new ArgumentException("متن پیام الزامیست");

// ارسال پیامک...
;Console.WriteLine($"SMS from {senderName} to {phoneNumber}: {message}")
;Console.WriteLine($"Sent at: {sendDate}, Urgent: {isUrgent}, Lang: {language}")

{

    ,public void ScheduleSms(string phoneNumber, string message
    ,string senderName, DateTime sendDate
    ,bool isUrgent, string language
    (DateTime scheduledTime
    }
```

```
// دوباره همون اعتبارسنجی ها!
if (string.IsNullOrEmpty(phoneNumber))
    throw new ArgumentException("شماره موبایل الزامیست");
if (string.IsNullOrEmpty(message))
    throw new ArgumentException("متن پیام الزامیست");

// زمان بندی پیامک ...
;Console.WriteLine($"Scheduled SMS from {senderName} to {phoneNumber}")
{
    {
```

مشکلات:

- ۶ تا پارامتر که ۶ تاش بین دو متد مشترک (Data Clumps)!
- اعتبارسنجی تکراری
- اگه بخوایم hasAttachment اضافه کنیم، باید همه جا رو تغییر بدیم

بعد از Introduce Parameter Object

csharp

```
// قدم ۱: ساختن کلاس پارامتر
public class SmsDetails
{
    public string PhoneNumber { get; set; }
    public string Message { get; set; }
    public string SenderName { get; set; }
    public DateTime SendDate { get; set; }
    public bool IsUrgent { get; set; }
    public string Language { get; set; }

    // اعتبارسنجی هم میاد توی کلاس خودش
    public void Validate()
    {
        if (string.IsNullOrEmpty(PhoneNumber))
            throw new ArgumentException("شماره موبایل الزامیست");
        if (string.IsNullOrEmpty(Message))
            throw new ArgumentException("متن پیام الزامیست");
    }
}
```

```
// قدم ۲: استفاده از شیء جدید
public class NotificationService
{
    public void SendSms(SmsDetails details)
    {
        details.Validate();

        Console.WriteLine($"SMS from {details.SenderName} to {details.PhoneNumber}: {details.Message}");
        Console.WriteLine($"Sent at: {details.SendDate}, Urgent: {details.IsUrgent}, Language: {details.Language}");
    }

    public void ScheduleSms(SmsDetails details, DateTime scheduledTime)
    {
        details.Validate();

        Console.WriteLine($"Scheduled SMS from {details.SenderName} to {details.PhoneNumber}");
    }
}
```



مزایا

مزیت	توضیح
امضای متد ساده‌تر	از ۶-۷ پارامتر به ۱-۲ پارامتر
جلوگیری از تکرار	اعتبارسنجی فقط یک جا (توی کلاس)
اضافه کردن فیلد جدید آسون‌تر	فقط به کلاس SmsDetails اضافه کن، امضای متدها تغییر نمی‌کنه
Data Clumps درمان میشه	داده‌هایی که با هم سفر می‌کردن، حالا به خونه دارن
کمک به Extract Method	پارامترهای کمتر = استخراج متد آسون‌تر

تمرین برای تو

این کد رو با **Introduce Parameter Object** ریفتکتور کن:

csharp

```

public class CustomerService
{
    ;()private readonly List<Customer> _customers = new

    )public void RegisterCustomer
        ,string firstName
        ,string lastName
        ,string email
        ,string phone
        ,string address
        ,string city
        ,string postalCode
        (DateTime birthDate
    }

    // اعتبارسنجی
    if (string.IsNullOrEmpty(firstName))
        ;("نام الزامیست")throw new ArgumentException
        if (string.IsNullOrEmpty(lastName))
        ;("نام خانوادگی الزامیست")throw new ArgumentException
        if (string.IsNullOrEmpty(email))
        ;("ایمیل الزامیست")throw new ArgumentException

    var customer = new Customer
    }

    ,FirstName = firstName
    ,LastName = lastName
    ,Email = email
    ,Phone = phone
    ,Address = address
    ,City = city
    ,PostalCode = postalCode
    BirthDate = birthDate
    ;{

    ;customers.Add(customer)_
    ;("$مشری {firstName} {lastName} ثبت شد")Console.WriteLine
    {

```

```

        )public void UpdateCustomer
            ,int customerId
            ,string firstName
            ,string lastName
            ,string email
            ,string phone
            ,string address
            ,string city
            ,string postalCode
            (DateTime birthDate
        }

        // دوباره همون اعتبارسنجی تکراری!
        if (string.IsNullOrEmpty(firstName))
            ;("نام الزامیست")throw new ArgumentException
        if (string.IsNullOrEmpty(lastName))
            ;("نام خانوادگی الزامیست")throw new ArgumentException
        if (string.IsNullOrEmpty(email))
            ;("ایمیل الزامیست")throw new ArgumentException

;var customer = _customers.FirstOrDefault(c => c.Id == customerId)
    ;if (customer == null) return

    ;customer.FirstName = firstName
    ;customer.LastName = lastName
    ;customer.Email = email
    ;customer.Phone = phone
    ;customer.Address = address
    ;customer.City = city
;customer.PostalCode = postalCode
;customer.BirthDate = birthDate

;("$مشتري {firstName} {lastName} بروزرسانی شد")Console.WriteLine
    {
    {

        public class Customer
        {

            public int Id { get; set; }
            public string FirstName { get; set; }
            public string LastName { get; set; }
            public string Email { get; set; }
            public string Phone { get; set; }
            public string Address { get; set; }
            public string City { get; set; }

```

```
public string PostalCode { get; set; }  
public DateTime BirthDate { get; set; }  
  
{
```

راهنمایی:

- به کلاس جدید مثل CustomerInfo یا CustomerDetails بساز
- ۸ پارامتر رو ببر توش
- اعتبارسنجی رو هم منتقل کن به کلاس جدید
- متدهای RegisterCustomer و UpdateCustomer رو بازنویسی کن

بفرست کدت رو تا بررسی کنیم! 📩